



Rúbrica de Evaluación Final

Proyecto DS4PS

Esta rúbrica detalla los requisitos técnicos y funcionales esperados para la entrega final del proyecto. Los ítems están clasificados por prioridad para guiar el desarrollo durante el próximo mes.

1. Arquitectura y Backend (FastAPI)

- **Estructura Modular [Prioridad Alta]:** El código no debe estar centralizado únicamente en `main.py`. Se requiere el uso de *routers* y la separación de la lógica en módulos independientes (ej. `database.py`, `esquemas.py`, rutas y estrategias).
- **Validación de Datos [Prioridad Alta]:** Uso correcto de Pydantic para validar los datos de entrada (tipos de datos, rangos lógicos, campos obligatorios).

2. Base de Datos

- **Persistencia en la Nube [Prioridad Alta]:** Migración del motor local SQLite a una base de datos relacional persistente en la nube (ej. PostgreSQL mediante Supabase o Neon.tech) para evitar la pérdida de datos en cada reinicio del servidor.
- **Modelado ORM [Prioridad Media]:** Uso de SQLAlchemy para mapear las tablas y realizar consultas de forma programática y limpia.

3. Seguridad y Autenticación

- **Sistema de Login [Prioridad Alta]:** Implementación de una pantalla de inicio de sesión en el frontend que requiera usuario y contraseña.
- **Protección de Endpoints [Prioridad Alta]:** Uso de JWT (JSON Web Tokens) o gestión de sesiones en FastAPI para bloquear el acceso a los endpoints a usuarios no autenticados.
- **Seguridad de Contraseñas [Prioridad Alta]:** Las contraseñas en la base de datos no deben guardarse en texto plano. Deben estar encriptadas (ej. utilizando la librería `passlib` con `bcrypt`).



4. Machine Learning / Datos

- **Modelo Predictivo [Prioridad Alta]:** Reemplazar las fórmulas matemáticas estáticas (ej. *Riesgo* = $x * y$) por un modelo entrenado (ej. Scikit-Learn) que prediga la prioridad, el riesgo o el costo del operativo basándose en datos históricos.
- **Serialización de Modelos [Prioridad Alta]:** El modelo de Machine Learning debe estar guardado previamente (en formato `.pkl` o `.joblib`) y cargarse en memoria al iniciar la API para realizar inferencias en tiempo real.

5. Frontend y UX/UI (Streamlit)

- **Navegación Multisección [Prioridad Alta]:** Uso de `st.sidebar` y `st.navigation` (o equivalente) para estructurar la aplicación en páginas separadas (ej. 'Login', 'Carga de Zonas', 'Dashboard Analítico').
- **Manejo de Estado [Prioridad Alta]:** Mantener al usuario logueado utilizando `st.session_state` para evitar que deba ingresar sus credenciales en cada interacción.
- **Visualización Avanzada [Prioridad Media]:** Incorporación de gráficos interactivos (mediante Plotly o Altair), mapas geoespaciales o métricas claras orientadas directamente a la toma de decisiones.

6. Despliegue (Deploy)

- **Backend en Producción [Prioridad Alta]:** La API de FastAPI debe estar desplegada y funcionando online en Render (o servicio similar) con una URL pública activa.
- **Frontend en Producción [Prioridad Alta]:** El dashboard interactivo debe estar publicado y accesible a través de Streamlit Community Cloud.
- **Gestión de Secretos [Prioridad Alta]:** Ninguna contraseña, token o URI de base de datos debe estar *hardcodeada* en el código fuente. Todo debe manejarse mediante Variables de Entorno (`.env` en local, *Environment Variables* en la nube).

7. Buenas Prácticas de Desarrollo

- **Archivo Requirements [Prioridad Alta]:** Presencia de un archivo `requirements.txt` completo y funcional que permita la instalación de todas las dependencias en los servidores de la nube.
- **Documentación (README) [Prioridad Alta]:** Un archivo `README.md` estructurado que explique el propósito del proyecto, las instrucciones paso a paso para ejecutarlo en local y las credenciales de prueba para la corrección.
- **Repositorio de GitHub [Prioridad Media]:** Uso adecuado del control de versiones. Commits descriptivos y un archivo `.gitignore` bien configurado (ignorando explícitamente el `.env` y las carpetas `__pycache__`).